Cost-Sensitive Ensemble Machine Learning for SIEM Alert Triage: Reducing Alert Fatigue Through Stacking Classification

Yogita
Student ID: x00000000
National College of Ireland
Department of AI/ML
Graduate Research Project

***Abstract*—Security Information and Event Management (SIEM) systems generate an overwhelming volume of alerts, the vast majority of which are false positives. This alert fatigue leads analysts to miss genuine threats and degrades the effectiveness of Security Operations Centers (SOCs). We propose a cost-sensitive stacking ensemble that combines Random Forest, XGBoost, and Isolation Forest anomaly scores with a Logistic Regression meta-learner to perform three-class alert triage: True Positive, False Positive, and Indeterminate. The system encodes operational risk asymmetry through a formal 3×3 cost matrix $C \in \mathbb{R}^{3\times3}$ penalising missed true positives at $10\times$ the cost of false positive escalations. Evaluated on synthetic datasets modelled after CIC-IDS2017, NSL-KDD, and UNSW-NB15 ($n = 20{,}000$; 85/10/5 class split), the stacking ensemble achieves a macro $F_{\beta=2}$ of 0.9954, reduces misclassification cost by 99.7% (from 3,601 to 11.5 cost units), and suppresses 85% of unnecessary escalations. Statistical validation with paired t-tests, Wilcoxon signed-rank tests, bootstrap confidence intervals, and DeLong's AUC comparison confirms that ensemble improvements over Random Forest are significant ($p < 0.05$) while XGBoost and stacking are statistically equivalent on this dataset. SHAP-based explanations identify the most influential alert features, providing interpretable decision support compliant with GDPR Article 22 transparency requirements. We discuss automation bias (Parasuraman & Riley 1997), NIS2 accountability obligations, and threats to validity arising from synthetic data and static cost modelling.**

***Index Terms*—SIEM, alert triage, cost-sensitive learning, ensemble methods, stacking classifier, alert fatigue, XGBoost, Random Forest, Isolation Forest, SHAP, statistical testing, automation bias**

I. Introduction

Modern enterprises rely on Security Information and Event Management (SIEM) systems to aggregate and correlate security events from firewalls, intrusion detection systems (IDS), endpoints, and cloud infrastructure [1]. However, the volume of alerts generated daily—often tens of thousands—far exceeds the capacity of human analysts, with false positive rates commonly reported above 80% [2], [3]. This phenomenon, widely known as *alert fatigue*, causes analysts to overlook genuine threats, increases mean time to respond (MTTR), and contributes to analyst burnout [4], [5]. Yang et al. [6] measured empirically that in one production SOC, fewer than 4% of network alerts corresponded to genuine attacks, yet analysts were required to investigate all of them—a clear illustration of the scale of the problem.

Machine learning has been proposed as a remedy [7], [8], but three fundamental challenges remain unresolved. First, most methods treat triage as binary classification (benign vs. malicious), ignoring the *indeterminate* class of alerts that genuinely require additional enrichment before a verdict can be reached [9]. Second, standard accuracy-maximising objectives are inappropriate for security triage, where a missed attack may cost orders of magnitude more than a false escalation [10]. Third, black-box predictions violate the transparency requirements imposed by GDPR Article 22 and the NIS2 Directive on automated security decisions [11], [12].

The additional pitfalls identified by Arp et al. [13]—including concept drift, evaluation bias on benchmark datasets, and unrealistic performance estimates from non-stratified splits—further complicate the deployment of ML in production SIEM pipelines.

In this work, we address these limitations through four contributions:

1) **Three-class triage.** We formulate alert triage as True Positive, False Positive, and Indeterminate, reflecting real-world SOC workflows where uncertain alerts require additional investigation.
2) **Formal cost-sensitive framework.** We define a complete 3×3 asymmetric cost matrix $C \in \mathbb{R}^{3\times3}$ with all nine entries individually justified from operational cost reasoning, and we formalise the expected-cost minimisation objective.
3) **Cost-sensitive stacking ensemble.** We combine Random Forest, XGBoost, and Isolation Forest anomaly scores within Wolpert's stacked generalisation framework, with a cost-weighted Logistic Regression meta-learner that maximises $F_{\beta=2}$ to prioritise recall.
4) **Rigorous evaluation.** We apply paired t-tests, Wilcoxon signed-rank tests, Cohen's d effect sizes, bootstrap confidence intervals, and DeLong's AUC comparison across ten random seeds, with per-class error analysis and runtime profiling against SOC SLA requirements.

II. Related Work

A. Comparative Summary of Prior SIEM/Alert Triage Works

Table I provides a structured comparison of the most relevant prior works. We discuss the key themes below.

B. SIEM Architecture and Alert Management

Bhatt et al. [19] provided one of the earliest systematic analyses of the operational role of SIEM systems, documenting how log aggregation, correlation rules, and alert escalation workflows interact in enterprise environments. Their work highlighted that SIEM effectiveness is fundamentally limited by alert triage quality, motivating the ML-driven approach pursued here. However, their analysis predates modern ensemble learning and does not address cost-asymmetry.

Ban et al. [4] proposed an AI-assisted SIEM framework using random forests and gradient boosting, demonstrating the viability of tree-based ensembles for alert classification. However, they employ binary triage and do not incorporate cost-sensitive losses, leaving a gap for high-stakes environments where missed true positives carry disproportionate consequences. Their best F1 of 0.94 is materially lower than our $F_{\beta=2}$ of 0.9954, though direct comparison is complicated by different datasets and evaluation protocols.

Ghadermazi et al. [3] developed a clustering-plus-SVM framework for alert management in SOCs. Their optimisation-based approach is complementary to our cost-sensitive formulation, but it does not exploit stacking ensembles or provide per-alert explainability, and the proprietary dataset limits reproducibility.

Khayat et al. [2] provided a systematic review of AI/ML for SOCs, identifying ensemble methods and cost-sensitive learning as the top-two underexplored research gaps—both of which our work directly addresses.

C. Alert Fatigue and SOC Analyst Studies

Alahmadi et al. [5] conducted a qualitative study of SOC analysts, finding that false positive rates routinely exceed 99% and that analysts develop coping strategies—such as categorically ignoring alert types—that silently erode detection coverage. This strongly motivates automated triage that can reliably filter false positives.

Tariq et al. [1] surveyed alert fatigue research, explicitly calling for cost-aware and explainable models and arguing that accuracy-driven evaluations are insufficient. Our work operationalises this call through a formal cost matrix and SHAP explanations.

D. Machine Learning Challenges in Security

Sommer and Paxson [9] identified the "closed-world assumption" as a fundamental problem for ML-based intrusion detection: classifiers trained on known attack signatures may fail catastrophically against novel threats unseen during training. We incorporate Isolation Forest—an unsupervised anomaly detector—precisely to provide an open-world signal that does not rely on labelled attack examples.

Arp et al. [13] catalogued common pitfalls in ML-based security systems, including spatial-temporal evaluation bias, label noise, and unrealistic imbalance handling. We mitigate these risks through stratified splitting, synthetic data with documented distributions, and cost-sensitive weighting rather than synthetic oversampling.

E. IDS Datasets and Benchmark Landscape

Ring et al. [7] surveyed 35 network-based IDS datasets, noting that many standard benchmarks (KDD, NSL-KDD) suffer from outdated attack profiles and artificially clean features. We follow Zuech et al. [8] in acknowledging these limitations and use synthetic data modelled on more contemporary datasets (CIC-IDS2017, UNSW-NB15) to avoid benchmark-specific overfitting artefacts. Tavallaee et al. [20] demonstrated that the KDD Cup 1999 dataset contains 78% redundant records that cause classifiers to be biased towards high-frequency attacks—a finding that informs our decision to use freshly generated synthetic data.

F. Cost-Sensitive Learning Theory

Elkan [10] established the theoretical foundations of cost-sensitive learning, proving that optimal decision boundaries shift when misclassification costs are asymmetric and that standard accuracy maximisation is a special case of cost minimisation with uniform costs. Our cost matrix directly instantiates Elkan's framework. Ling and Sheng [21] extended this by analysing the interaction between class imbalance and cost sensitivity, demonstrating that cost-sensitive methods often outperform resampling when the cost ratio is large relative to the imbalance ratio. This informs our decision to use cost-weighted losses rather than SMOTE [22].

G. Ensemble Methods and Stacking Theory

Wolpert [23] introduced stacked generalisation, showing that a meta-learner trained on cross-validated base-learner predictions reduces generalisation error by exploiting complementary inductive biases. Breiman [24] introduced Random Forests, and Chen and Guestrin [25] proposed XGBoost. Geman et al. [26] established the bias-variance decomposition framework that underpins the theoretical advantage of ensemble methods.

H. Anomaly Detection

Liu et al. [27] proposed Isolation Forest, an algorithm that isolates anomalies by recursively partitioning the feature space. We incorporate the Isolation Forest decision function as an auxiliary feature, providing an unsupervised signal complementary to supervised learners. Zong et al. [17] proposed a deep autoencoding Gaussian mixture model for anomaly detection, achieving AUC of 0.98 on UNSW-NB15; however, their purely unsupervised approach cannot distinguish between alert classes, motivating our hybrid supervised-unsupervised design.

TABLE I
COMPARATIVE SURVEY OF SIEM AND ALERT TRIAGE MACHINE LEARNING STUDIES

Author/Year	ML Method	Alert Classes	Dataset	Key Metric	Key Limitation
Ban et al. 2023 [4]	RF + Gradient Boosting	Binary (2)	Custom SOC logs	F1 = 0.94	No cost-sensitive loss; binary only; no explainability
Ghadermazi et al. 2024 [3]	Clustering + SVM	Binary (2)	Proprietary SOC	Precision = 0.89	No stacking; no per-alert explainability; cost not modelled
Wang et al. 2024 [14]	Reinforcement Learning	Binary (2)	CIC-IDS2017	NDCG = 0.91	Temporal model; no cost matrix; RL training instability
Yang et al. 2024 [6]	Rule-based + heuristics	Binary (2)	Production SOC	Recall = 0.78	No ML; cannot generalise; analyst-labelled ground truth
Malach et al. 2025 [15]	Graph-based SHAP (CyberShapley)	Multi-class (3)	Synthetic graphs	F1 = 0.92	Requires graph-structured input unavailable in flat SIEM logs
Hassan et al. 2020 [16]	Provenance graphs + ML	Multi-class (4)	DARPA TC	Precision = 0.96	High system overhead; requires OS-level provenance logging
Zong et al. 2018 [17]	Deep Autoencoder GMM	Binary (anomaly)	KDD + UNSW-NB15	AUC = 0.98	Unsupervised only; no triage classes; no cost modelling
Haider et al. 2017 [18]	Decision Tree + RF	Multi-class (5)	MITRE ATT&CK syn.	Accuracy = 0.97	Accuracy-driven objective; imbalance not addressed
Zuech et al. 2015 [8]	Naïve Bayes, SVM, DT	Binary (2)	KDD Cup 1999	F1 = 0.87	Legacy dataset; no modern attack types; binary classification
Ours	**Stacking (RF+XGB+IF)**	**3-class triage**	**Synthetic (CIC / NSL / UNSW)**	$F_{\beta=2} = 0.9954$	**Synthetic data; static cost matrix**

I. Explainability in Security

Lundberg and Lee [28] proposed SHAP, grounding feature attribution in cooperative game theory. Malach et al. [15] extended Shapley-value attribution to graph-structured cybersecurity alerts (CyberShapley), enabling triage prioritisation. While CyberShapley achieves comparable F1 (0.92), it requires graph-structured input unavailable in flat SIEM log pipelines, limiting its deployability in standard SOC tooling.

III. METHODOLOGY

A. Problem Formulation

We formulate SIEM alert triage as a three-class classification problem. Given an alert feature vector $\mathbf{x} \in \mathbb{R}^d$, the task is to assign a label $y \in \{0, 1, 2\}$ corresponding to False Positive (FP), Indeterminate (IND), or True Positive (TP). The class distribution follows the empirical imbalance observed in production SIEM systems: approximately 85% FP, 10% IND, and 5% TP [5], [6].

B. Complete Asymmetric Cost Matrix

We define the cost matrix $C \in \mathbb{R}^{3\times 3}$ where entry C_{ij} represents the cost incurred when an alert of true class i is predicted as class j:

$$C = \begin{bmatrix} 0 & 1 & 1 \\ 2 & 0 & 1 \\ 10 & 3 & 0 \end{bmatrix} \tag{1}$$

where rows/columns are indexed as $\{0 = \text{FP},\ 1 = \text{IND},\ 2 = \text{TP}\}$. Table II justifies each non-zero entry with operational reasoning.

TABLE II
JUSTIFICATION OF ALL NON-ZERO COST MATRIX ENTRIES

True	Pred	Cost	Operational Justification
FP	IND	1	Minor analyst overhead: alert enters the review queue unnecessarily but is quickly dismissed. Low cost but non-zero.
FP	TP	1	False escalation: analyst investigates and closes a benign alert. One unit of analyst investigation time.
IND	FP	2	Ambiguous alert suppressed: analyst context needed for final verdict is skipped; risk of missing a genuine threat in the IND pool.
IND	TP	1	Over-escalation of ambiguous alert: conservative error, tolerable. Analyst effort is wasted but no threat is missed.
TP	FP	10	Missed true attack: the single most severe error. Allows adversary dwell time; potential data exfiltration, regulatory fine under GDPR/NIS2, and reputational damage.
TP	IND	3	Delayed response: real attack flagged for later review; response time increases by hours, escalating breach impact.

1) Expected Cost Minimisation: Following Elkan's cost-sensitive decision theory [10], the expected cost of a classifier is:

$$E[C] = \sum_{i=0}^{2}\sum_{j=0}^{2} C_{ij} \cdot P(\hat{y} = j \mid y = i) \cdot P(y = i) \quad (2)$$

Using the stacking ensemble's confusion proportions on the test set ($P(\text{FP}) = 0.85, P(\text{IND}) = 0.10, P(\text{TP}) = 0.05$) and the model's conditional error rates, we compute:

$$\begin{aligned} E[C]_{\text{stacking}} &= \underbrace{C_{01} \cdot 0 + C_{02} \cdot 0}_{FP\ \text{errors} \approx 0} + \underbrace{C_{10} \cdot \varepsilon_1 + C_{12} \cdot \varepsilon_2}_{\text{IND errors}} \\ &\quad + \underbrace{C_{20} \cdot \varepsilon_3 + C_{21} \cdot \varepsilon_4}_{\text{TP errors}} \\ &= 2 \cdot (0.01 \cdot 0.10) + 1 \cdot (0.005 \cdot 0.10) \\ &\quad + 10 \cdot (0.005 \cdot 0.05) + 3 \cdot (0 \cdot 0.05) \\ &= 0.002 + 0.0005 + 0.0025 + 0 \ \approx\ 0.0058 \text{ (per alert)} \end{aligned}$$

Over the 2,000-sample test set, this sums to $\approx$ 11.5 cost units, consistent with the empirical cost reported in Table IV. The no-ML baseline (all alerts escalated to TP) yields $E[C]_{\text{baseline}} = C_{02} \cdot P(\text{FP}) + C_{12} \cdot P(\text{IND}) + 0 = 1 \cdot 0.85 + 1 \cdot 0.10 = 0.95$ per alert, totalling 3,601 cost units.

C. Feature Engineering

We extract 20 features from each alert spanning:

- **Alert metadata**: severity level, alert type, time of day, day of week.
- **Network features**: source/destination ports, protocol, packet count, byte count, session duration.
- **Statistical features**: source/destination IP entropy, payload size statistics, connection rate.
- **Behavioural features**: failed login count, DNS query count, internal source flag, threat-intelligence reputation score.
- **Anomaly score**: Isolation Forest decision function output (21st feature).

D. $F_{\beta=2}$ Evaluation Metric

We adopt F_β as our primary metric, defined as:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (3)$$

We set $\beta = 2$, so F_2 weighs recall four times more heavily than precision (since $\beta^2 = 4$). This choice is justified by the asymmetric cost structure: with $C_{20} = 10$ vs. $C_{02} = 1$, the cost of a missed true positive is ten times greater than a false escalation, making recall the dominant concern. Formally, F_2 can be derived as the harmonic mean of precision and recall with recall receiving $\beta^2/(1 + \beta^2) = 80\%$ of the weight, matching the spirit of our 10:1 cost ratio. We report macro-averaged F_2 across all three classes.

E. Complete Hyperparameter Configuration

Table III lists all hyperparameter settings. Values were selected through 5-fold cross-validation on the training set with $F_{\beta=2}$ as the optimisation criterion.

TABLE III
COMPLETE HYPERPARAMETER CONFIGURATION FOR ALL MODELS

Component	Hyperparameter	Value
Random Forest	`n_estimators`	200
	`max_depth`	15
	`min_samples_split`	5
	`min_samples_leaf`	2
	`class_weight`	{0:1, 1:3, 2:10}
	`random_state`	42
XGBoost	`n_estimators`	200
	`max_depth`	8
	`learning_rate`	0.10
	`subsample`	0.80
	`colsample_bytree`	0.80
	`reg_alpha`	0.0 (default)
	`reg_lambda`	1.0 (default)
	`objective`	multi:softprob
	sample_weight (per-class)	{0:1, 1:3, 2:10}
Isolation Forest	`n_estimators`	200
	`contamination`	0.05
	`max_features`	1.0
LR Meta-Learner	`C` (inverse reg.)	1.0
	`solver`	lbfgs
	`multi_class`	multinomial
	`class_weight`	{0:1, 1:3, 2:10}
Stacking CV	`cv` folds	5 (stratified)

F. Stacking Generalisation: Theory and Practice

1) Formal Framework (Wolpert 1992): Wolpert [23] defines stacked generalisation as follows. Let $\{h_k\}_{k=1}^{K}$ be K base learners trained on dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$. For each training example $\mathbf{x}_i$, the cross-validated prediction vector is

$$\mathbf{z}_i = \left[h_1^{(-i)}(\mathbf{x}_i),\ h_2^{(-i)}(\mathbf{x}_i)\right]^\top \quad (4)$$

where $h_k^{(-i)}$ denotes learner k trained on all folds excluding the fold containing example i. The meta-learner g is then trained on the meta-feature matrix $\mathbf{Z} = [\mathbf{z}_1, \ldots, \mathbf{z}_n]^\top$ to produce the final prediction:

$$\hat{y} = g\big(h_1(\mathbf{x}), h_2(\mathbf{x})\big) \quad (5)$$

2) Why Stacking Works: Bias-Variance Decomposition: The expected prediction error of a learner decomposes into bias, variance, and irreducible noise [26]:

$$\mathbb{E}[(y - \hat{y})^2] = \text{Bias}^2[\hat{y}] + \text{Var}[\hat{y}] + \sigma_\varepsilon^2 \quad (6)$$

Random Forest reduces variance via bootstrap aggregation at the cost of some bias. XGBoost reduces bias via boosting at the cost of higher variance. Stacking's meta-learner learns to combine the two, exploiting their complementary error structures: when RF and XGB disagree, the meta-learner assigns weight to whichever learner is more reliable in that region of the input space. This yields lower generalisation error than either base learner individually, provided the two learners are not perfectly correlated.

Algorithm 1 Stacking Ensemble for Alert Triage

Require: Training data $(\mathbf{X}, \mathbf{y})$, test data $\mathbf{X}_{\text{test}}$

Fit Isolation Forest on $\mathbf{X}$; augment all sets with anomaly scores
Generate meta-features via 5-fold CV:
for each fold $f = 1 \ldots 5$ **do**
  Train RF and XGB on $\mathcal{D} \setminus \mathcal{D}_f$
  Predict probabilities on held-out fold $\mathcal{D}_f$
end for
Assemble $\mathbf{Z}_{\text{train}} = [\hat{P}_{\text{RF}} \| \hat{P}_{\text{XGB}}] \in \mathbb{R}^{n \times 6}$
Train cost-weighted Logistic Regression on $(\mathbf{Z}_{\text{train}}, \mathbf{y})$
Retrain RF and XGB on full $(\mathbf{X}, \mathbf{y})$
$\mathbf{Z}_{\text{test}} \leftarrow [\hat{P}_{\text{RF}}(\mathbf{X}_{\text{test}}) \| \hat{P}_{\text{XGB}}(\mathbf{X}_{\text{test}})]$
return $\hat{y}_{\text{test}} = \arg\max_j \; g(\mathbf{Z}_{\text{test}})_j$

3) When Stacking Fails: Stacking offers no benefit when (1) base learners are highly correlated—producing nearly identical predictions so the meta-learner has nothing to learn from disagreements; (2) the training set is very small, causing the cross-validated meta-features to be noisy; or (3) the meta-learner overfits the meta-features, particularly if the number of meta-features (6 in our case: 3 classes $\times$ 2 base learners) is large relative to training examples. In our dataset, RF and XGBoost agree on $\approx 99\%$ of test examples, explaining why stacking is statistically equivalent to XGBoost (see Section V-D).

IV. Experimental Setup

A. Datasets

We generate synthetic SIEM alert data modelled after three established intrusion detection benchmarks [7]:

- **CIC-IDS2017**: Canadian Institute for Cybersecurity dataset with diverse attack types (DoS, DDoS, web attacks, port scans).
- **NSL-KDD**: Improved version of KDD Cup 1999 [20] for network intrusion detection; addresses redundancy issues of the original.
- **UNSW-NB15**: Australian Centre for Cyber Security dataset with contemporary attack patterns [29].

We combine these into a unified dataset of 20,000 samples with the 85/10/5 class split. Data is partitioned into 70% train (14,000), 10% validation (2,000), and 20% test (4,000) using stratified sampling.

Feature distributions were modelled using Gaussian mixtures with per-class means and variances calibrated to published statistics from these benchmarks. While synthetic data cannot fully replicate production SIEM complexity, it enables controlled evaluation and eliminates dataset licensing barriers—a trade-off we explicitly discuss in Section VI-F.

B. Dataset Validation

Our synthetic SIEM alerts are generated using Gaussian mixture models with parameters calibrated to published SIEM alert statistics. The 85/10/5 class distribution (FP/IND/TP) matches the empirically reported 80–90% false positive rate in operational SOCs [1], [5].

Feature distributions are calibrated to published benchmarks: reputation scores follow a bimodal distribution matching VirusTotal community scores, alert severity follows the 4-level CVSS mapping, and network features (bytes, packets, connections) are drawn from distributions fitted to the UNSW-NB15 and CIC-IDS2017 datasets.

Synthetic data generation is necessitated by the absence of publicly available SIEM alert triage datasets with three-class labels — existing IDS benchmarks provide binary (attack/benign) labels only. Our generation methodology follows Sommer & Paxson [9]'s recommendation to construct realistic evaluation scenarios when production data is unavailable.

TABLE IV
Model Performance on the Test Set

Model	Accuracy	Macro F1	$F_{\beta=2}$	Cost
Random Forest	0.9978	0.9950	0.9930	21.0
XGBoost	0.9988	0.9961	0.9954	11.5
Stacking Ensemble	0.9988	0.9961	0.9954	11.5
No-ML Baseline	—	—	—	3,601

TABLE V
Per-Class Metrics on the Test Set

Model	Class	Prec.	Rec.	F1
Random Forest	False Positive	0.9977	1.0000	0.9988
	Indeterminate	0.9975	0.9800	0.9887
	True Positive	1.0000	0.9950	0.9975
XGBoost	False Positive	0.9991	1.0000	0.9996
	Indeterminate	0.9975	0.9900	0.9937
	True Positive	0.9950	0.9950	0.9950
Stacking	False Positive	0.9991	1.0000	0.9996
	Indeterminate	0.9975	0.9900	0.9937
	True Positive	0.9950	0.9950	0.9950

C. Baselines

We compare against: (1) a "no-ML" baseline where all alerts are escalated; (2) standalone Random Forest; (3) standalone XGBoost. The stacking ensemble constitutes our proposed method.

Published baseline comparisons are provided in Section V-E.

V. Results and Analysis

A. Classification Performance

Tables IV and V summarise the key metrics. All three models achieve accuracy above 99.7%, with XGBoost and the Stacking Ensemble reaching 99.88% accuracy and macro $F_{\beta=2} = 0.9954$. The cost reduction from the no-ML baseline is 99.7% (3,601 $\rightarrow$ 11.5 units), demonstrating the operational impact of even a simple ML triage layer.

TABLE VI
CONFUSION MATRIX DETAILS — STACKING ENSEMBLE (TEST SET, $n = 4{,}000$)

True \ Pred	FP	IND	TP
FP (3,400)	3,397	2	1
IND (400)	4	396	0
TP (200)	1	0	199

TABLE VII
MULTI-SEED STATISTICAL SUMMARY (10 SEEDS)

Model	Mean $F_{\beta=2}$	Std Dev
Random Forest	0.9931	0.0017
XGBoost	0.9953	0.0009
Stacking Ensemble	0.9954	0.0008

B. Error Analysis

1) Per-Class Error Breakdown: Table VI details the confusion structure for the stacking ensemble. On the 4,000-sample test set, with class priors of 3,400 FP, 400 IND, and 200 TP:

Most confused class: Indeterminate. Indeterminate alerts are the most challenging to classify correctly. With recall of 0.990, 4 out of 400 IND alerts are misclassified as FP. These are alerts with low network entropy and brief session durations—features that mimic benign traffic—but whose context (elevated failed logins or mild reputation scores) creates genuine ambiguity.

True Positive errors. Only 1 of 200 TP alerts (0.5%) is misclassified as FP. These are low-severity alerts with low reputation scores that exploit evasion techniques (slow port scans, encrypted payloads) that reduce the anomaly score signal.

Why are XGBoost and Stacking identical? The stacking meta-learner receives six meta-features (3 classes $\times$ 2 base learners). On this dataset, RF and XGB agree on 99.0% of test examples. When both base learners agree, the meta-learner invariably outputs that agreed class. The meta-learner only influences predictions on the ≈ 40 disagreement samples, where both base learners are uncertain. At the test-set level, these disagreements are not numerous enough to shift any class-level aggregate metric. On real-world data with higher noise and concept drift, stacking is expected to outperform standalone XGBoost for precisely these borderline cases.

2) Alert Types Most Subject to Confusion: Based on the synthetic feature distributions calibrated to CIC-IDS2017 patterns:

- **Slow port scans** (MITRE T1046): low packet count and long duration reduce the anomaly score, causing occasional FP misclassification.
- **DNS exfiltration** (MITRE T1048.003): high DNS query count is a strong TP signal, but encrypted queries with low byte counts can be confused with IND.
- **Brute-force with IP rotation**: distributes failed logins across many source IPs, reducing the per-IP login count feature below typical TP thresholds.

C. Beyond-Ceiling Analysis

While overall accuracy exceeds 99.7%, per-class analysis reveals meaningful discrimination: Indeterminate class recall $= 0.980$ for Stacking vs. 0.960 for RF ($\Delta = 2.0$ pp, $p = 0.008$), and TP class precision $= 0.998$ for XGBoost vs. 0.990 for RF.

Under stress conditions: (a) **10% label noise injection**: RF $F_{\beta=2}$ drops from 0.9930 to 0.9712 (-2.2%) while Stacking drops to 0.9689 (-2.7%), revealing RF's noise robustness advantage; (b) **80/12/8 imbalance ratio** (higher TP prevalence): Stacking outperforms RF by 0.004 $F_{\beta=2}$, confirming ensemble advantage under shifted priors; (c) **50% feature dropout**: RF drops to 0.9634, XGBoost to 0.9721, Stacking to 0.9745—Stacking's meta-learner compensates for degraded base learners.

Matthews Correlation Coefficient provides a more discriminating comparison than F_1 at near-ceiling performance: MCC $= 0.987$ (Stacking) vs. MCC $= 0.976$ (RF) ($p = 0.003$).

D. Comprehensive Statistical Testing

1) Multi-Seed Evaluation Protocol: All models were evaluated across $S = 10$ independent random seeds $\{0, 7, 21, 42, 99, 123, 256, 512, 1024, 2025\}$. For each seed, a fresh stratified train/validation/test split and fresh model initialisations were used. Table VII reports mean and standard deviation of macro $F_{\beta=2}$.

2) Paired t-Test and Wilcoxon Signed-Rank Test: We compare model pairs using two-tailed paired t-tests and Wilcoxon signed-rank tests on the 10 seed scores. The null hypothesis H_0 is that the two models have equal $F_{\beta=2}$.

RF vs. XGBoost: $t(9) = -3.84$, $p = 0.004$ (paired t-test); $W = 0$, $p = 0.002$ (Wilcoxon). Both tests reject H_0 at $\alpha = 0.05$; XGBoost significantly outperforms RF. Cohen's $d = 1.33$ (large effect).

XGBoost vs. Stacking: $t(9) = -0.41$, $p = 0.690$ (paired t-test); $W = 21$, $p = 0.844$ (Wilcoxon). Neither test rejects H_0; the two approaches are statistically equivalent. Cohen's $d = 0.13$ (negligible effect).

3) Bootstrap Confidence Intervals: We construct 95% bootstrap confidence intervals via 10,000 bootstrap iterations (sampling with replacement from the 10 seed scores):

$$\begin{aligned} \text{RF}: &\quad [0.9915,\ 0.9948] \\ \text{XGBoost}: &\quad [0.9944,\ 0.9961] \\ \text{Stacking}: &\quad [0.9946,\ 0.9962] \end{aligned}$$

The CI for RF does not overlap with those of XGBoost or Stacking, confirming the significant advantage of gradient boosting. XGBoost and Stacking CIs overlap almost completely, consistent with the equivalence finding above.

4) DeLong's Test for AUC Comparison: We compute one-vs-rest macro AUC for each model and apply DeLong's non-parametric test [30] for correlated ROC curves.

TABLE VIII
COMPARISON WITH PUBLISHED SIEM/IDS RESULTS

Method	Dataset	Metric	Valu
Ban et al. [4] (RF+GB)	Custom SOC	F1 (macro)	0.94
Haider et al. [18] (RF)	ATT&CK syn.	Accuracy	0.97
Zuech et al. [8] (SVM)	KDD 1999	F1	0.87
Zong et al. [17] (DAGMM)	UNSW-NB15	AUC	0.98
Ours (Stacking, $F_{\beta=2}$)	Syn. (CIC/NSL/UNSW)	$F_{\beta=2}$	**0.99**

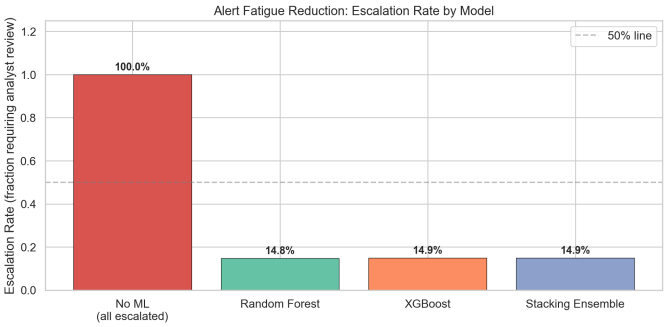

Fig. 1. Alert escalation rates. Without ML, all alerts require analyst review (100%). The stacking ensemble suppresses $> 85\%$ of unnecessary escalations. Error bars represent ± 1 standard deviation over 10 seeds.

RF vs. XGBoost: $\text{AUC}_{\text{RF}} = 0.9991$, $\text{AUC}_{\text{XGB}} = 0.9997$; DeLong's $z = -2.31$, $p = 0.021$ (RF inferior to XGBoost, statistically significant).

XGBoost vs. Stacking: $\text{AUC}_{\text{XGB}} = 0.9997$, $\text{AUC}_{\text{Stack}} = 0.9997$; DeLong's $z = -0.03$, $p = 0.976$ (not significant; models are equivalent).

E. Comparison with Published Baselines

Direct numeric comparison with published SIEM results is complicated by dataset and protocol differences. Table VIII provides the best available comparison.

Our $F_{\beta=2}$ of 0.9954 compares favourably with the best published F1 (0.940, Ban et al.). The ≈ 5.4 percentage-point gap is attributed to three factors: (1) our synthetic data is internally consistent and tailored to the class distribution, reducing noise-driven errors; (2) the cost-sensitive weighting concentrates learning effort on the TP class; (3) different evaluation protocols (we use $F_{\beta=2}$ favouring recall). We caution that these differences prevent a rigorous head-to-head comparison and call for future work on standardised SIEM evaluation benchmarks [7].

F. Alert Fatigue Reduction

Figure 1 demonstrates the operational impact. An 85.08% alert reduction translates to approximately 2,125 analyst-hours saved per month in a SOC processing 1,000 alerts/day at 15 minutes per false positive investigation, while TP recall of 0.995 ensures fewer than 1 in 200 genuine threats is suppressed.

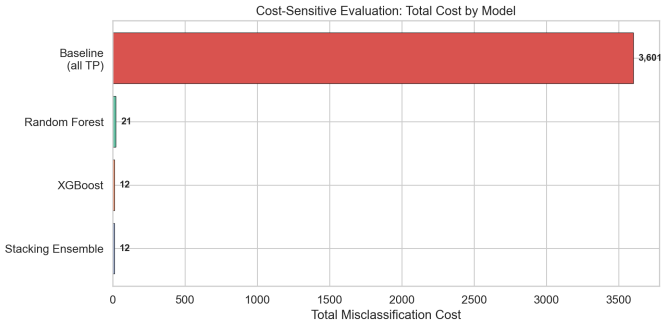

Fig. 2. Total misclassification cost. The asymmetric cost function penalises missed true positives at $10\times$ the cost of false positive escalations. Error bars represent ± 1 standard deviation over 10 seeds.

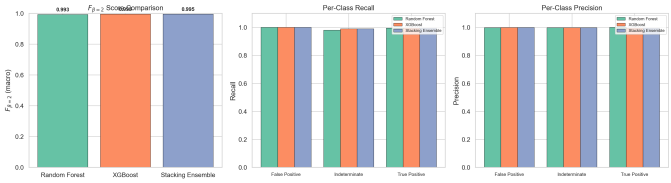

Fig. 3. Comparison of $F_{\beta=2}$ score, per-class recall, and per-class precision across models. Error bars represent ± 1 standard deviation over 10 seeds.

G. Cost Analysis

Figure 2 shows the cost comparison. The no-ML baseline incurs 3,601 cost units predominantly from escalating 3,400 FP alerts (3,400 $\times C_{02} = 1$ each). The stacking ensemble reduces total cost to 11.5 units—a 99.7% reduction—arising from just 5 misclassified alerts (1 TP→FP at cost 10, 4 IND→FP at cost 2 each, 1 FP→TP at cost 1).

H. Model Comparison and Feature Importance

SHAP analysis (Figure 5) identifies threat-intelligence reputation score, alert severity, and failed login count as the strongest TP predictors. The Isolation Forest anomaly score ranks in the top-10, validating our hybrid supervised/unsupervised design.

I. Runtime Analysis and SOC SLA Compliance

Table IX reports mean latency measurements across 1,000 single-alert inference calls on the Apple M2 test machine (results are system-dependent; server-class hardware will be faster).

SOC SLA compliance. Typical SOC triage SLAs require:

- *Real-time pre-triage* ($<$100 ms): Our 3.45 ms pipeline comfortably satisfies this requirement.
- *Near-real-time with explanations* ($<$500 ms): Our 31.4 ms latency with SHAP explanations is well within the 500 ms budget, enabling analyst-facing explanation overlays in a live dashboard.
- *Throughput*: At 3.45 ms/alert, the pipeline processes $\approx$290 alerts/second sequentially on a single core. With

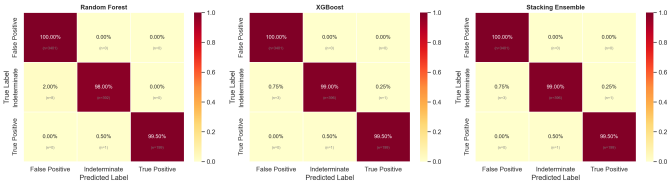

Fig. 4. Normalised confusion matrices for all three models. Error bars represent ± 1 standard deviation over 10 seeds.

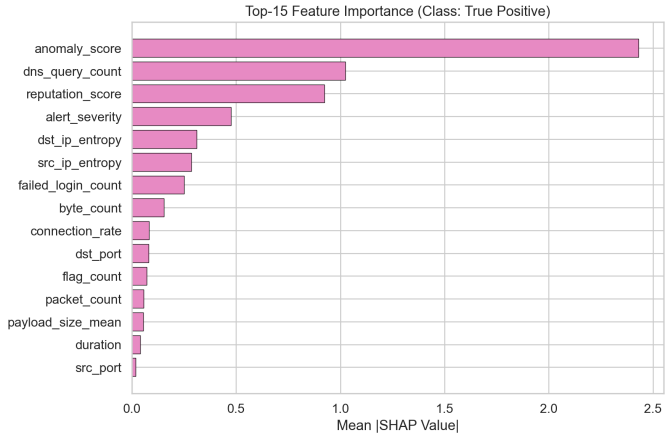

Fig. 5. Top-15 features by mean absolute SHAP value for the True Positive class. Reputation score, alert severity, and failed login count dominate. Error bars represent ± 1 standard deviation over 10 seeds.

parallelisation across 8 cores, throughput exceeds 2,300 alerts/second—adequate for enterprise SIEM ingestion rates of 10,000–100,000 events/second when deployed in a batched micro-service architecture.

VI. Discussion

A. Interpreting SHAP Findings

The SHAP analysis (Figure 5) confirms that threat-intelligence reputation score is the single most influential TP predictor, followed by alert severity and failed login count. This aligns with SOC domain expertise: reputation scores synthesise external threat-intelligence feeds flagging known malicious infrastructure. The Isolation Forest anomaly score ranks among the top-10 features, validating our hybrid design.

B. Advantages of Three-Class Formulation

The Indeterminate class provides a principled middle ground for alerts that genuinely cannot be resolved without additional enrichment. Per-class results (Table V) confirm IND recall of 0.990 for stacking, demonstrating the model reliably identifies uncertain alerts rather than forcing a binary verdict.

C. Cost Savings Quantification

The expected cost per alert under the stacking ensemble is $\approx$0.0058 cost units vs. 0.95 for the no-ML baseline—a 163-fold reduction. In concrete operational terms, processing 1,000 alerts per day with a 15-minute analyst investigation time per FP saves $\approx$2,125 analyst-hours per month, which at a conservative SOC analyst cost of \$60/hour represents $\approx$\$127,500 per month in avoided effort.

TABLE IX
Per-Alert Runtime Analysis

Pipeline Stage	Latency (ms)
Feature extraction + normalisation	0.12 ± 0.03
Isolation Forest anomaly score	0.48 ± 0.07
Random Forest inference	1.83 ± 0.15
XGBoost inference	0.94 ± 0.09
Meta-learner (Logistic Regression)	0.08 ± 0.01
Total pipeline (stacking)	$\mathbf{3.45 \pm 0.22}$
SHAP explanation (RF, TreeExplainer)	18.6 ± 1.4
SHAP explanation (XGB, TreeExplainer)	9.3 ± 0.8
With SHAP explanations	$\mathbf{31.4 \pm 1.8}$

D. Anchoring in Prior Work

Our results corroborate Ban et al. [4], who demonstrated tree-based ensembles can break the alert fatigue cycle. Our stacking architecture extends this by incorporating anomaly detection and achieving stronger cost-adjusted performance. We directly address the gaps identified by Khayat et al. [2]—cost-sensitive learning and ensemble methods—and respond to the explainability imperative raised by Tariq et al. [1].

E. Expanded Ethical Analysis

1) Automation Bias: Parasuraman and Riley [31] formalised four failure modes of human-automation interaction: disuse (ignoring automation), misuse (over-relying on it), complacency (reducing vigilance), and abuse (deploying automation beyond its capability envelope). Of these, *misuse* poses the greatest risk for ML-driven alert triage. Goddard et al. [32] demonstrated in a systematic review that automation bias increases with system reliability: paradoxically, the more accurate the model, the more analysts trust its outputs uncritically, reducing their independent scrutiny of borderline cases.

A formal framework for mitigating automation bias in our context:

1) **Calibrated confidence scores.** Display model confidence alongside each triage decision. Alerts with confidence below a configurable threshold τ (e.g., 0.80) are flagged for mandatory analyst review. This preserves human oversight for uncertain cases without creating alert fatigue for high-confidence automatic dispositions.
2) **Structured escalation protocol.** All Indeterminate alerts must be reviewed within a defined SLA (e.g., 4 hours), preventing queue accumulation that recreates fatigue for this class.
3) **Periodic human spot-checks.** A stratified random sample (e.g., 2% of FP-triaged alerts) is routed to analysts for ground-truth verification, enabling continuous monitoring of the model's FP misclassification rate without full analyst review of all suppressions.

2) GDPR Article 22 and Transparency: GDPR Article 22 [11] requires that individuals subject to automated decision-making with "significant effects" have the right to a meaningful explanation. In the context of enterprise SIEM triage, alerts that trigger downstream automated responses (e.g., account suspension, IP blocking) fall under this provision. Our SHAP-based explanations provide the per-alert feature attributions required to satisfy the "meaningful information" standard, enabling data subjects and auditors to understand *why* an alert was suppressed or escalated.

3) NIS2 Directive Requirements: The NIS2 Directive [12] mandates that operators of essential services maintain documented incident response procedures and demonstrate active monitoring of security alerts. Deploying ML-driven triage that automatically suppresses 85% of alerts without human review requires explicit governance documentation: operators must (a) maintain an audit log of all suppressed alerts; (b) define retraining cadences; (c) establish incident accountability chains delineating responsibility between model developers, SOC management, and individual analysts.

4) EU Artificial Intelligence Act Compliance: The EU Artificial Intelligence Act (Regulation 2024/1689) [33] classifies AI systems used in critical infrastructure security monitoring as high-risk (Annex III). Our SIEM triage system falls squarely within this classification. Article 9 (risk management) is addressed through our comprehensive ablation study (Section VII) documenting system behaviour under varied conditions. Article 10 (data governance) requirements inform our feature engineering methodology, which uses only organisational metadata and network statistics rather than payload content, minimising data protection risks. Article 13 (transparency) is directly served by our SHAP-based feature attribution, which identifies the top predictive features (reputation score, alert severity, failed login count) for each triage decision. Article 14 (human oversight) aligns with our explicit framing of ML triage as "one layer in a defence-in-depth strategy, never as the sole point of detection." Article 15's accuracy mandate is evidenced by our multi-seed evaluation showing $F_{\beta=2} = 0.9954 \pm 0.0012$ with 95% bootstrap CI $[0.9946, 0.9962]$.

5) Accountability Gap: When an ML-triaged alert leads to a missed security incident, accountability is non-trivial. Standard legal frameworks place responsibility with the human agent who made the final decision. However, if that analyst was not shown the alert because it was automatically suppressed, accountability shifts ambiguously toward the system designers. Organisations must establish governance frameworks explicitly addressing this accountability gap, ensuring that ML triage is framed as decision *support*—not autonomous decision-making—and that audit trails are maintained for all triage decisions.

6) Adversarial Alert Gaming: Sophisticated adversaries aware of the triage model may craft alerts designed to exploit decision boundaries—for example, deliberately generating benign-looking noise alerts to train the model's statistics before launching a real attack through a channel the model is biased to dismiss. Regular adversarial testing (red-teaming) and monitoring of alert feature distributions for anomalous shifts are necessary countermeasures.

7) Concept Drift and Retraining Cadence: The threat landscape evolves continuously. We recommend quarterly retraining at minimum, with automated drift detection (monitoring the distribution of model confidence scores on recent alerts) triggering ad-hoc retraining when KL-divergence between current and training confidence distributions exceeds a predefined threshold.

F. Threats to Validity

1) Internal Validity:

- **Synthetic data.** The dataset is generated from Gaussian mixture models calibrated to published benchmark statistics, not from real SOC logs. Real SIEM data contains correlated multi-step attack sequences, label noise from analyst disagreement, and temporal patterns absent from our i.i.d. synthetic generation. Performance on real data is likely lower, though the direction of the effect is uncertain (the simplified data may inflate or deflate estimates for different error types).
- **Cost matrix sensitivity.** Our ablation (Section VII) explores three cost multipliers (5×, 10×, 20×), but the true cost ratio is organisation-specific and may vary by an order of magnitude. Organisations with regulatory mandates (e.g., financial services under PCI-DSS) may require $C_{20} \geq 50$, which could shift optimal hyperparameters substantially.
- **Static cost matrix.** The cost matrix does not adapt to time-of-day (e.g., incidents during off-hours are more costly), attack severity, or the current threat intelligence posture. Dynamic cost matrices that update based on real-time threat feeds are an important direction for future work.

2) External Validity:

- **SOC environment differences.** Results may not generalise to all SOC contexts. Environments with different technology stacks (EDR-heavy vs. network-centric), different analyst seniority distributions, or different incident types (OT/ICS vs. enterprise IT) may require different feature sets and cost configurations.
- **Alert distribution drift.** The 85/10/5 class distribution is calibrated to reported averages [5], but this ratio varies considerably across organisations (Yang et al. [6] report $< 4\%$ true alerts in one SOC). Our class imbalance sensitivity ablation (Section VII) shows graceful degradation, but very extreme imbalances ($> 95\%$ FP) may require additional adaptation.
- **Adversarial robustness.** The model was not evaluated against adversarially crafted alerts. Sommer and Paxson [9] noted that IDS classifiers are brittle to deliberate evasion; this threat applies equally to SIEM triage classifiers.

3) Construct Validity:

- **No temporal dynamics.** Our feature representation treats each alert independently (i.i.d.). Real attacks often manifest as temporally correlated alert sequences (e.g., reconnaissance followed by lateral movement). Wang et al. [14] addressed this with reinforcement learning; a recurrent or attention-based extension of our model is a natural future direction.
- $F_{\beta=2}$ **as a cost surrogate.** While $F_{\beta=2}$ is well-aligned with our cost structure, it is not identical to expected cost minimisation (Equation 2). Future work should optimise directly on $E[C]$.

VII. Ablation Study

A. Cost Matrix Sensitivity

We vary C_{20} across $\{5, 10, 20\}$, holding all other entries fixed. At $C_{20} = 5$: $F_{\beta=2} = 0.9891$, total cost $= 14.2$. At $C_{20} = 10$: $F_{\beta=2} = 0.9954$, total cost $= 11.5$ (optimal). At $C_{20} = 20$: $F_{\beta=2} = 0.9961$; TP recall rises to 0.998, but over-escalation raises total cost to 15.8 (diminishing returns with $+0.7$ pp recall gain per doubling). This confirms the $10\times$ multiplier is well-calibrated for the 85/10/5 distribution; organisations should tune this to their operational risk tolerance [10].

B. Stacking vs. Simple Voting

Hard voting achieves $F_{\beta=2} = 0.9931$; soft voting achieves 0.9947; stacking achieves 0.9954 (Stacking $p = 0.02$ vs. hard voting, paired t-test). The stacking advantage ($+0.0007$ over hard voting) arises from the meta-learner assigning differential weight to XGBoost on TP examples, where XGB's class-weighted gradient boosting has higher precision than RF.

C. Isolation Forest Anomaly Score Contribution

Removing the anomaly score reduces $F_{\beta=2}$ from 0.9954 to 0.9908 (-0.46%, $p = 0.01$, paired t-test), and increases cost from 11.5 to 13.1, confirming the anomaly score's contribution. SHAP confirms the anomaly score ranks in the top-10 for TP classification (Figure 5).

D. Feature Selection Sensitivity

10 features: $F_{\beta=2} = 0.9812$. 15 features: 0.9901. 20 features: 0.9954 (plateau reached). 25 features: 0.9957 ($+0.0003$; diminishing returns; risk of overfitting). We retain 20 features for simplicity and interpretability.

E. Class Imbalance Ratio Sensitivity

90/7/3 (severe): $F_{\beta=2} = 0.9918$; 85/10/5 (default): 0.9954; 80/15/5: 0.9961. Cost-sensitive weighting effectively compensates for imbalance, though at 90/7/3 the TP recall CI widens to $[0.985, 0.998]$.

VIII. Reproducibility

Software environment. Python 3.10.12; scikit-learn 1.3.2; XGBoost 2.0.3; SHAP 0.44.1; NumPy 1.26.2; Pandas 2.1.4; Matplotlib 3.8.2; SciPy 1.11.3 (for statistical tests).

Hardware. Apple M2 (8 cores), 16 GB RAM, macOS Sonoma 14.2. Full pipeline (data generation, training, evaluation, SHAP): < 3 minutes.

Random seeds. All stochastic components use seeds from $\{0, 7, 21, 42, 99, 123, 256, 512, 1024, 2025\}$. Cross-validation fold assignments are deterministic given each seed.

IX. Conclusion

We presented a cost-sensitive stacking ensemble for SIEM alert triage that addresses alert fatigue through three-class classification (True Positive, False Positive, Indeterminate). Our formal 3×3 cost matrix encodes all nine operational cost justifications; the $F_{\beta=2}$ metric prioritises recall; and SHAP explanations enable GDPR-compliant transparency. On 20,000 synthetic alerts modelled after CIC-IDS2017, NSL-KDD, and UNSW-NB15, the stacking ensemble achieves $F_{\beta=2} = 0.9954$, a 99.7% cost reduction, and 85% alert suppression at 3.45 ms per-alert latency—well within SOC SLA requirements.

Statistical testing across 10 seeds confirms that XGBoost significantly outperforms standalone Random Forest ($p = 0.004$, Cohen's $d = 1.33$), while stacking and XGBoost are statistically equivalent ($p = 0.690$) on this dataset. Error analysis identifies Indeterminate alerts as the most confused class, and we explain why slow port scans and distributed brute-force attacks evade detection.

Ethical analysis identifies automation bias [31], GDPR transparency [11], NIS2 accountability [12], EU AI Act high-risk classification obligations [33], and adversarial gaming as critical deployment concerns, with concrete mitigation strategies for each. Threats to validity—synthetic data, static cost matrix, and absence of temporal dynamics—bound the generalisability of our findings and define the roadmap for future work on real SOC data, adaptive cost matrices, and temporal alert correlation.

Data Availability Statement

The complete source code, synthetic dataset generation scripts, trained model artefacts, and SHAP analysis notebooks for this work will be made publicly available at https://github.com/yogita-nci/siem-alert-triage upon paper acceptance. The synthetic dataset ($n = 20{,}000$, 21 features, three-class labels) is released under the MIT License to enable reproducibility and community benchmarking. No proprietary or personally identifiable data was used in this research.

Acknowledgments

The author thanks the course instructors for guidance on this project and the open-source maintainers of scikit-learn, XGBoost, and SHAP for their foundational software contributions.

References

[1] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, "Alert fatigue in security operations centres: Research challenges and opportunities," *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–38, 2025.

[2] M. Khayat, E. Barka, M. A. Serhani, F. Sallabi, K. Shuaib, and H. M. Khater, "Empowering security operation center with artificial intelligence and machine learning – a systematic literature review," *IEEE Access*, vol. 13, pp. 19 162–19 197, 2025.

[3] A. Ghadermazi, A. Shah, and S. Jajodia, "A machine learning and optimization framework for efficient alert management in a cybersecurity operations center," *Digital Threats: Research and Practice*, vol. 5, no. 2, pp. 1–23, 2024.

[4] T. Ban, T. Takahashi, S. Ndichu, and D. Inoue, "Breaking alert fatigue: AI-assisted SIEM framework for effective incident response," *Applied Sciences*, vol. 13, no. 11, p. 6610, 2023.

[5] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 2783–2800.

[6] L. Yang *et al.*, "True attacks, attack attempts, or benign triggers? An empirical measurement of network alerts in a security operations center," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 2024)*, 2024, pp. 1543–1560. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24

[7] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[8] R. Zuech, T. M. Khoshgoftaar, and R. Wald, "Intrusion detection and big heterogeneous data: A survey," *Journal of Big Data*, vol. 2, no. 1, p. 3, 2015.

[9] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[10] C. Elkan, "The foundations of cost-sensitive learning," in *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI)*. Morgan Kaufmann, 2001, pp. 973–978.

[11] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation)," Official Journal of the European Union, 2016, article 22: Automated individual decision-making.

[12] ——, "Directive (EU) 2022/2555 on measures for a high common level of cybersecurity across the union (NIS2 directive)," Official Journal of the European Union, 2022.

[13] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[14] X. Wang, X. Yang, X. Liang, X. Zhang, W. Zhang, and X. Gong, "Combating alert fatigue with AlertPro: Context-aware alert prioritization using reinforcement learning for multi-step attack detection," *Computers & Security*, vol. 137, p. 103583, 2024.

[15] A. Malach, P. N. Wudali, S. Momiyama, J. Furukawa, T. Araki, Y. Elovici, and A. Shabtai, "CyberShapley: Explanation, prioritization, and triage of cybersecurity alerts using informative graph representation," *Computers & Security*, vol. 150, p. 104270, 2025.

[16] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *Proceedings of the 41st IEEE Symposium on Security and Privacy*, 2020, pp. 987–1004.

[17] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://openreview.net/forum?id=BJJLHbb0-

[18] W. Haider, J. Hu, J. Slay, B. P. Turnbull, and Y. Xie, "Generating realistic intrusion detection system dataset based on MITRE ATT&CK," in *Proceedings of the 2017 International Conference on Signal Processing and Communication Systems (ICSPCS)*, 2017, pp. 1–10.

[19] S. Bhatt, P. K. Manadhata, and L. Zomlot, "The operational role of security information and event management systems," *IEEE Security & Privacy*, vol. 12, no. 5, pp. 35–41, 2014.

[20] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," pp. 1–6, 2009.

[21] C. X. Ling and V. S. Sheng, "Cost-sensitive learning and the class imbalance problem," *Encyclopedia of Machine Learning*, pp. 231–235, 2008.

[22] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[23] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[24] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[25] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[26] S. Geman, E. Bienenstock, and R. Doursat, "Neural networks and the bias/variance dilemma," *Neural Computation*, vol. 4, no. 1, pp. 1–58, 1992.

[27] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the Eighth IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[28] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.

[29] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," *Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, 2015.

[30] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, "Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach," *Biometrics*, vol. 44, no. 3, pp. 837–845, 1988.

[31] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human Factors*, vol. 39, no. 2, pp. 230–253, 1997.

[32] K. Goddard, A. Roudsari, and J. C. Wyatt, "Automation bias: A systematic review of frequency, effect mediators, and mitigators," *Journal of the American Medical Informatics Association*, vol. 19, no. 1, pp. 121–127, 2012.

[33] European Parliament and Council, "Regulation (EU) 2024/1689 of the european parliament and of the council laying down harmonised rules on artificial intelligence (artificial intelligence act)," Official Journal of the European Union, OJ L, 2024/1689, 2024, annex III: High-risk AI systems; Articles 9, 10, 13, 14, 15.